WIRELESS NETWORK SECURITY · APPLIED MACHINE LEARNING

Detecting Wi-Fi Deauthentication Attacks with Machine Learning

A lightweight intrusion detection & prevention system (WIDS / WIPS) that learns normal 802.11 behaviour from a few simple features, then flags — and blocks — a deauthentication flood in real time.

Author: Ramla Abdullahi **Student number:** 190169

1 INTRODUCTION

A deauthentication attack abuses an unprotected 802.11 management frame. By spoofing the access point's address, an attacker repeatedly tells clients they have been disconnected — knocking devices off the network. It is cheap, common, and often the first step in a larger attack such as an evil-twin or handshake capture.

Aim: read wireless traffic, learn normal vs attack from a handful of simple features, and raise an alert — or automatically block the offender — with no hand-written rules.

Why ML, not a fixed threshold? Legitimate deauth frames do occur (a client leaving). A model weighs *rate, frame mix and signal behaviour* together instead of firing on every single deauth.

2 DATASET

20 labelled capture scenarios (~20 s each) were built with **Scapy**, modelling a small lab network: an access point, 2–5 clients browsing and streaming, handshakes and power-save frames. Every frame carries a RadioTap **signal-strength** reading and a ground-truth label.

- **Normal traffic includes legitimate deauth / disassoc frames**, so the model cannot cheat with "any deauth = attack".
- **Attacks:** broadcast flood, targeted flood, low-rate stealth flood, disassociation flood, rogue-MAC flood.
- Structured to mirror the public **AWID3** dataset, so the pipeline can retrain on real traces.

Lab-only & ethical: all frames are generated in memory and written to files — nothing is transmitted over the air. Live capture or attack generation is confined to an authorised, isolated testbed.

3 HOW IT WORKS

An attack is not one odd frame — it is an abnormal *burst* from one device. Frames are grouped into **1-second windows per transmitter**, each described by 14 simple numbers, before a classifier decides.

STEP 1 · CAPTURE
Read 802.11 frames
A .pcap file or a live monitor-mode interface — same code path.

STEP 2 · FEATURES
Per device, per second
Frame rates, type mix, signal strength, deauth & broadcast share.

STEP 3 · CLASSIFY
Random Forest
Labels each window normal or malicious (scikit-learn).

STEP 4 · RESPOND
Alert → Block
WIDS logs an alert; WIPS blocklists the offending MAC.

Features: `rssi_std` `frac_broadcast` `deauth_per_sec` `frames_per_sec` `frac_mgmt` `frac_deauth` `n_unique_dst` + 7 more

4 RESULTS

Tested by holding out **whole scenarios** from training (a group split) — scored on attacks it never saw. No leakage.

99.5%	1.00
ACCURACY	PRECISION · ATTACK
0.88	**0**
RECALL · ATTACK	FALSE ALARMS

A single Decision Tree managed 0.75 recall; the Random Forest lifts it to **0.88** with the same zero false alarms.

CONFUSION MATRIX · HELD-OUT TEST

ACTUAL \ PRED.	PRED. NORMAL	PRED. ATTACK
ACTUAL NORMAL	421 TRUE NORMAL	0 FALSE ALARM
ACTUAL ATTACK	2 MISSED	14 CAUGHT

The 2 misses are the first / last second of a short attack, when few flood frames fall in the window.

TOP FEATURES

rssi_std	.23
frac_broadcast	.19
deauth_per_sec	.11
frac_mgmt	.10
frames_per_sec	.08

5 LIVE DEMO

Real-time replay: normal traffic scrolls past, then the flood spikes `deauth/s`, the classifier fires, and WIPS blocks the attacker — while a real client is untouched.

```
 t   device            d/s   status
10  1c:2e:…:9d          0   ok
11  1c:2e:…:9d        103   ATTACK
    └ WIPS blocked 1c:2e:…:9d
12  1c:2e:…:9d        144   ATTACK
13  10:0e:…:c2          0   ok (real client)
14  1c:2e:…:9d        143   ATTACK
─────────────────────────────────
Mitigated. rule applied:
iptables -A INPUT --mac-source
  1c:2e:…:9d -j DROP
```

6 CONCLUSION

A few explainable features and an off-the-shelf Random Forest reliably separate a deauth flood from normal Wi-Fi — **99.5% accuracy, no false alarms** on unseen attacks. Prevention mode turns detection into an automatic, logged response in under a second.

FUTURE WORK

- Train and validate on the full **AWID3** dataset and authorised live captures.
- Go **multi-class** — name the attack, not just normal vs malicious.
- Overlapping windows to catch the first second of an attack (today's 2 misses).

ACKNOWLEDGEMENTS & TOOLS

Built in Python with **Scapy** (capture & frame crafting), **scikit-learn** (Random Forest / Decision Tree) and **Matplotlib**.

AI assistance: a coding assistant helped scaffold the Python modules and draft this poster. The design decisions — per-device time-window features, keeping legitimate deauths in the normal class, and a leakage-free group split — were my own, and every result was verified by running the code.

REFERENCES

1. Bellardo & Savage (2003). *802.11 Denial-of-Service Attacks*. USENIX Security.
2. Chatzoglou, Kambourakis & Kolias (2021). *The AWID3 Dataset*. IEEE Access.
3. Pedregosa et al. (2011). *Scikit-learn*. JMLR 12.

REPRODUCE

One command:

```
pip install -r requirements.txt && bash run_all.sh
```

Generates data, trains, evaluates and runs the WIDS + WIPS demos.

Code & poster:
`github.com/rumiixii/WIDS-Project`